Gu[?][?]a del Worker de Scraping

- InmoOS Scraper Worker — Guía completa de despliegue
  - 1. Arquitectura final acordada
  - 2. Pre-requisitos en tu servidor
  - 3. Estructura del repo del worker
  - 4. Variables de entorno
  - 5. Despliegue en Coolify (paso a paso)
  - 6. Contrato HTTP
  - 7. Buenas prácticas anti-detección (críticas)
  - 8. Operativa
  - 9. Riesgo legal
  - 10. Checklist final
- Anexo: Despliegue en Coolify
- Coolify — Setup detallado para el worker de InmoOS
  - 1. Instalar Coolify (una vez por servidor)
  - 2. Crear el proyecto
  - 3. Añadir Redis
  - 4. Generar token de API
  - 5. Obtener el UUID de la Application
  - 6. Webhook de redespliegue automático
  - 7. Logs y debugging
  - 8. Auto-update del worker desde Git

InmoOS Scraper Worker — Guía completa de despliegue

Worker externo que realiza búsquedas reales en Idealista, Fotocasa y Habitaclia para alimentar el módulo **Oportunidades** de InmoOS.

> **Por qué vive fuera de Lovable / Supabase:** los portales bloquean IPs de datacenter (Cloudflare, DataDome, PerimeterX) y exigen un navegador

> real con fingerprint humano. Edge Functions no pueden ejecutar Playwright ni mantener sesiones largas con proxies residenciales.

---

1. Arquitectura final acordada

```
┌──────────────────────┐   crea job   ┌──────────────────────────┐
│  Lovable (UI)        │ ──────────── ► │  Edge: scraper-create-  │
│  /oportunidades      │              │  job                     │
└──────────────────────┘              └──────────────┬───────────┘
                                                     │ POST /jobs
                                                     ▼
┌──────────────────────────────────────────────────────────────────┐
│  WORKER COMPARTIDO MULTI-TENANT (tu servidor, vía Coolify)  │
│  ──────────────────────────────────────────────────────────────── │
│  Hono server (3000)  ◄── BullMQ ──► Redis  ──► Playwright   │
│                                  + Stealth      │
│                                  + Proxy rot.   │
└──────────────────────────────────────┬──────────────────────────┘
                                       │ POST resultados
                                       ▼
                          ┌──────────────────────────────────┐
                          │  Edge: scraper-ingest-results    │
                          │  → tabla scraper_results         │
                          └──────────────────────────────────┘
```

- **Un único contenedor** procesa jobs de todos los tenants (filtrados por `tenant_id`).
- **Coolify** despliega el contenedor desde el repo Git que crearás con este código.
- **Proxies** se inyectan como variables de entorno (cualquier proveedor HTTP/SOCKS5).
- **Heartbeat** cada 30s a `worker-heartbeat` para que la UI muestre estado online/offline.

---

2. Pre-requisitos en tu servidor

1. VPS Linux (Ubuntu 22.04+ recomendado), mínimo **4 vCPU / 8 GB RAM / 40 GB SSD**.

2. Dominio o subdominio apuntando al servidor (ej. `scraper.tudominio.com`).

3. Coolify instalado:

```
curl -fsSL https://cdn.coollabs.io/coolify/install.sh | sudo bash
```

4. Una cuenta en un proveedor de proxies residenciales (Bright Data, Smartproxy, IPRoyal…). Lo decides cuando quieras: el worker acepta cualquiera.

---

3. Estructura del repo del worker

Crea un repo Git nuevo (GitHub/GitLab) con esta estructura. Los ficheros completos están en `docs/scraper-worker/files/`.

```
scraper-worker/
├── Dockerfile
├── docker-compose.yml          # solo para desarrollo local
├── package.json
├── tsconfig.json
├── .env.example
├── nixpacks.toml               # opcional, si Coolify usa Nixpacks
└── src/
    ├── server.ts               # Hono HTTP server
    ├── queue.ts                # BullMQ + Redis
    ├── worker.ts               # Procesador de jobs
    ├── ingest.ts               # POST a scraper-ingest-results
    ├── heartbeat.ts            # Ping cada 30s a worker-heartbeat
    ├── proxy.ts                # Rotación + selección de proxy
    ├── browser.ts              # Playwright + stealth + fingerprint
    └── adapters/
        ├── idealista.ts
        ├── fotocasa.ts
        └── habitaclia.ts
```

---

4. Variables de entorno

```
# Identidad y comunicación con Lovable
SUPABASE_URL=https://wcnhjqfshcxqqauvxjhs.supabase.co
WORKER_TOKEN=<shared-secret>              # MISMO valor que en Lovable
WORKER_VERSION=1.0.0
WORKER_ID=worker-eu-1                     # libre, identifica este nodo

# Cola
REDIS_URL=redis://redis:6379

# Proxies (rellena cuando elijas proveedor)
```

```
PROXY_PROVIDER=brightdata              # brightdata | smartproxy | iproyal | custom
PROXY_HOST=brd.superproxy.io:22225
PROXY_USER=
PROXY_PASS=
PROXY_COUNTRY=es

# Comportamiento
MAX_CONCURRENT_JOBS=3
REQUEST_DELAY_MS=8000                  # entre requests al mismo dominio
JOB_TIMEOUT_MS=300000                  # 5 min por job
HEARTBEAT_INTERVAL_MS=30000
```

---

5. Despliegue en Coolify (paso a paso)

5.1 Crear el proyecto del worker

1. En Coolify → **New Resource** → **Application** → **Public/Private Git Repo**.
2. Pega la URL del repo del worker.
3. **Build Pack:** `Dockerfile` (o `Nixpacks` si lo prefieres).
4. **Port:** `3000`.
5. **Domain:** `scraper.tudominio.com` (Coolify gestiona el HTTPS con Let's Encrypt).
6. Pega todas las variables de entorno de la sección 4.
7. Pulsa **Deploy**.

5.2 Añadir Redis como servicio

1. En el mismo proyecto → **+ New Resource** → **Database** → **Redis**.
2. Coolify expone `redis://redis:6379` dentro de la red interna.

5.3 Generar Personal Access Token de Coolify

1. Coolify → tu avatar → **Keys & Tokens** → **Create New Token**.
2. Permisos: `read`, `write`, `deploy`.
3. Copia el token y el **UUID de la Application** del worker.

5.4 Pegar credenciales en Lovable

Ve a `/configuracion/worker` (página construida en la app) y pega: - **Coolify API URL:** `https://coolify.tudominio.com/api/v1` - **Coolify API Token:** el de 5.3 - **Application UUID:** el de la

app del worker - **Worker URL pública:** `https://scraper.tudominio.com` - **Worker Token:** el mismo `WORKER_TOKEN` de 4.

A partir de aquí, **Lovable redespliega el worker** vía API de Coolify cuando cambias proxies o config — sin SSH ni intervención manual.

---

6. Contrato HTTP

6.1 Recepción de un job

```
POST https://scraper.tudominio.com/jobs
X-Worker-Token: <shared-secret>
Content-Type: application/json

{
  "jobId": "uuid",
  "tenantId": "uuid",
  "params": {
    "operation": "compra | alquiler | alquiler_temporal",
    "propertyTypes": ["piso", "casa"],
    "city": "Madrid",
    "zones": ["Salamanca", "Chamberí"],
    "priceMin": 100000, "priceMax": 500000,
    "surfaceMin": 60, "surfaceMax": 150,
    "roomsMin": 2, "bathroomsMin": 1,
    "listingType": "particular | agencia | ambos",
    "features": ["ascensor", "terraza"],
    "adAge": "24h | 7d | 30d | any"
  },
  "portals": ["idealista", "fotocasa", "habitaclia"]
}
```

Respuesta inmediata: `{ "ok": true, "queued": true }` (HTTP 202).

6.2 Envío de resultados (streaming)

El worker llama a `scraper-ingest-results` cada vez que termina una página o detecta un lote de cards (cada 10–30s aprox.):

```
POST https://wcnhjqfshcxqqauvxjhs.supabase.co/functions/v1/scraper-ingest-results
X-Worker-Token: <shared-secret>
Content-Type: application/json
```

```
{
  "jobId": "uuid",
  "tenantId": "uuid",
  "results": [{ "portal": "idealista", "external_id": "...", ... }],
  "progress": { "idealista": { "status": "running", "count": 24 } },
  "status": "running"
}
```

Al terminar todos los portales: `{ "jobId": "uuid", "status": "done" }`. Si falla: `{ "jobId": "uuid", "status": "error", "error": "captcha_blocked" }`.

6.3 Heartbeat

```
 POST https://wcnhjqfshcxqqauvxjhs.supabase.co/functions/v1/worker-heartbeat
X-Worker-Token: <shared-secret>
Content-Type: application/json

{
  "workerId": "worker-eu-1",
  "version": "1.0.0",
  "queueDepth": 4,
  "activeJobs": 2,
  "metrics": {
    "jobsLast24h": 187,
    "successRate": 0.94,
    "avgLatencyMs": 42100
  }
}
```

---

7. Buenas prácticas anti-detección (críticas)

Riesgo	Mitigación
Fingerprint de navegador	`playwright-extra` + `puppeteer-extra-plugin-stealth`
IP de datacenter	Proxies **residenciales rotatorios** por sesión
Patrones robóticos	`humanScroll()` con easing, delays aleatorios 200–800ms

Riesgo	Mitigación
Cabeceras sospechosas	UA realistas pool de 30+ navegadores reales actualizados mensualmente
Cookies vacías	Persistir `storageState` por proxy 6–24h
429 / 403	Backoff exponencial: 1m → 5m → 30m → swap proxy
CAPTCHA	Detectar `iframe[src*="hcaptcha"]` / `recaptcha` → cambiar proxy + fingerprint
Geolocalización	`--lang=es-ES`, `timezoneId: "Europe/Madrid"`, proxy `country=es`
Rate por dominio	≤1 request cada 8s por proxy por portal

---

8. Operativa

- **Concurrencia recomendada:** 2–4 jobs simultáneos por instancia del worker.
- **Cuotas por tenant:** ≤30 jobs/hora (configurable en `worker_config`).
- **Cache por hash de query:** 6–24h (Redis con TTL).
- **Logs:** Coolify → tu app → tab **Logs** (real-time).
- **Métricas:** la página `/configuracion/worker` en Lovable las muestra en tiempo real.
- **Actualizaciones del worker:** push al repo → Coolify auto-deploy (webhook).

---

9. Riesgo legal

El scraping de portales infringe sus Términos de Uso. El cliente final asume el riesgo. InmoOS facilita la herramienta pero **no garantiza disponibilidad**: si un portal endurece sus defensas, los adapters deben actualizarse. Recomendamos:

- No revender los datos brutos.
- Respetar `robots.txt` cuando sea posible.
- Honrar opt-outs de anunciantes particulares.
- Tener consentimiento del cliente en los Términos del SaaS.

---

10. Checklist final

- [ ] Repo del worker creado con los ficheros de `docs/scraper-worker/files/`.
- [ ] Servidor con Coolify operativo y dominio apuntando.
- [ ] App + Redis desplegados en Coolify.
- [ ] Variables de entorno configuradas (incluido `WORKER_TOKEN`).
- [ ] Personal Access Token de Coolify generado.
- [ ] Credenciales pegadas en `/configuracion/worker` de Lovable.
- [ ] Heartbeat verde en la UI de Lovable.
- [ ] Job de prueba lanzado desde `/oportunidades` y resultados recibidos.
- [ ] Proveedor de proxies elegido y credenciales inyectadas.

Cuando los 9 primeros estén tildados, ya estás en producción real sin mocks.

Anexo: Despliegue en Coolify

Coolify — Setup detallado para el worker de InmoOS

1. Instalar Coolify (una vez por servidor)

```
curl -fsSL https://cdn.coollabs.io/coolify/install.sh | sudo bash
```

Accede a `http://<ip-servidor>:8000`, crea cuenta admin y configura tu dominio (ej. `coolify.tudominio.com`) con HTTPS automático.

2. Crear el proyecto

1. Coolify → **Projects** → **+ Add** → nombre: `inmoos`.
2. Dentro del proyecto → **+ New Resource** → **Application**.
3. **Source:** Public/Private Git Repository → URL del repo del worker.

4. **Build Pack:** `Dockerfile`.
5. **Branch:** `main`.
6. **Port:** `3000`.
7. **Domains:** `scraper.tudominio.com` (Coolify pide A record DNS apuntando al server).
8. **Environment Variables:** pega las de `.env.example` con valores reales.
9. **Deploy**.

3. Añadir Redis

1. Mismo proyecto → **+ New Resource** → **Database** → **Redis**.
2. Versión: 7.
3. Coolify lo expone internamente como `redis://redis:6379`.

4. Generar token de API

1. Avatar arriba derecha → **Keys & Tokens** → **API Tokens** → **Create New Token**.
2. Nombre: `lovable-inmoos`.
3. Permissions: marca **read** + **write** + **deploy**.
4. Copia el token (empieza por `1|...`). **No se vuelve a mostrar.**

5. Obtener el UUID de la Application

1. Abre la app del worker en Coolify.
2. Mira la URL: `https://coolify.tudominio.com/project/<proj-uuid>/environment/.../application/<APP-UUID>`.
3. El UUID de la app es el último segmento.

6. Webhook de redespliegue automático

Lovable usará el endpoint:

```
POST https://coolify.tudominio.com/api/v1/applications/<APP-UUID>/restart
Authorization: Bearer <coolify-token>
```

Para redeploys completos (cuando cambias env vars):

```
POST https://coolify.tudominio.com/api/v1/applications/<APP-UUID>/deploy
Authorization: Bearer <coolify-token>
```

Y para inyectar nuevas variables:

```
PATCH https://coolify.tudominio.com/api/v1/applications/<APP-UUID>/envs
Authorization: Bearer <coolify-token>
Content-Type: application/json

{ "key": "PROXY_PASS", "value": "...", "is_preview": false }
```

Estos tres calls son los que la Edge Function `worker-provision` ejecutará desde Lovable cuando pulses **Desplegar / Redesplegar / Reiniciar** en la UI.

7. Logs y debugging

- Coolify → tu app → **Logs** (real-time, últimas 1000 líneas).
- También vía API:

  ```
  GET https://coolify.tudominio.com/api/v1/applications/<APP-UUID>/logs
  ```

8. Auto-update del worker desde Git

Coolify → app → **Webhooks** → activa **Auto-deploy on push**. Cada vez que hagas push al repo del worker, Coolify reconstruye y despliega solo. Los adapters anti-detección se actualizan así sin tocar nada en Lovable.